# Minimum-Snap Planning and Measured Gate-Passing Performance in Quadrotor Racing

Junxi Li
Student ID: 24020036033 Group 6
Ocean University of China
Integrated Robotics Group Project

***Abstract*—This report develops polynomial trajectory optimisation for a twelve-gate quadrotor race. The mathematical exercises formulate minimum-velocity interpolation as a quadratic program and derive the boundary and continuity conditions for minimum-snap segments. The implemented planner generates position and yaw references, refines segment times under sampled velocity and acceleration limits, and evaluates references at incoming state timestamps. A geometric controller tracks the trajectory in Unity/TESSE. The final recording used a planned duration of $18.907\,\mathrm{s}$ and achieved a reported RMS tracking error of $0.0627\,\mathrm{m}$, a maximum nearest-gate-centre distance of $0.0771\,\mathrm{m}$, and a terminal position error of $0.0169\,\mathrm{m}$. My experimental analysis examines local approach accuracy, timestamp alignment and the acceleration demands associated with faster traversal.**

***Index Terms*—quadrotor, minimum snap, polynomial trajectory, waypoint planning, ROS, drone racing, trajectory tracking**

## I. Experiment Context and My Contribution

The Lab 4 exercise [1] connects analytic trajectory optimisation to closed-loop racing through the official Unity/TESSE environment. A polynomial describes the motion between prescribed gates; its derivatives provide the position, velocity and acceleration reference consumed by the controller. The three assessed deliverables concern single-segment optimisation, multi-segment optimisation, and the implementation and validation of a race-course trajectory.

My role was Member D: racing trajectory optimisation and experimental validation. I focused on the relationship between segment timing, tracking error and gate-centre proximity, and on the interpretation of the recorded flight. This report develops the mathematical solutions for D1 and D2, explains the implemented D3 system, and presents my analysis of the race data. The group implementation, configuration and recordings are available in the project repository [4].

## II. Deliverable 1: A Minimum-Velocity Path as a QP

### A. From an integral cost to a quadratic matrix

The first exercise asks for a real polynomial $P(t)$ on $0 \le t \le 1$ satisfying $P(0) = 0$, $P(1) = 1$ while minimising

$$J(P) = \int_0^1 [P'(t)]^2\, dt. \tag{1}$$

Write $P(t) = \sum_{i=1}^{N} p_i t^i$, with $p_0 = 0$ from the initial position constraint. Collecting like powers after squaring the derivative gives a quadratic program (QP):

$$J = \boldsymbol{p}^T Q \boldsymbol{p}, \qquad Q_{ij} = \frac{ij}{i+j-1}, \quad A\boldsymbol{p} = b, \tag{2}$$

where $A = [1, \ldots, 1]$ and $b = 1$. If a numerical solver uses $\frac{1}{2}\boldsymbol{p}^T H \boldsymbol{p}$, its Hessian must be $H = 2Q$.

### B. Answers for degrees one, two and three

For $N = 1$, the endpoint fixes $p_1 = 1$; hence $P^*(t) = t$ and $J^* = 1$. For $N = 2$, the requested QP matrices are

$$Q_2 = \begin{bmatrix} 1 & 1 \\ 1 & \frac{4}{3} \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 1 \end{bmatrix}, \quad b = 1. \tag{3}$$

Using $p_1 = 1 - p_2$ gives $J = 1 + p_2^2/3$. The optimum is $(p_1, p_2) = (1, 0)$, with the same linear polynomial and $J^* = 1$. For $N = 3$, the matrices become

$$Q_3 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & \frac{4}{3} & \frac{3}{2} \\ 1 & \frac{3}{2} & \frac{9}{5} \end{bmatrix}, \quad A_3 = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}, \quad b = 1. \tag{4}$$

The KKT conditions $2Q_3\boldsymbol{p} + A_3^T\lambda = 0$, $A_3\boldsymbol{p} = 1$ yield $\boldsymbol{p}^* = (1, 0, 0)^T$, $\lambda = -2$; therefore $P^*(t) = t$ and $J^* = 1$ again. The result agrees with the Euler–Lagrange equation $P''(t) = 0$. More generally, Cauchy–Schwarz provides

$$\int_0^1 (P')^2 dt \ge \left[\int_0^1 P' dt\right]^2 = [P(1) - P(0)]^2 = 1. \tag{5}$$

Constant velocity attains the bound for every permitted degree.

TABLE I
SCALAR CONDITIONS FOR $k$ MINIMUM-SNAP SEGMENTS.

| Condition | Count |
|---|---|
| Endpoint positions and both sides of every interior waypoint | $2k$ |
| Interior derivatives 1–6 match across each knot | $6(k-1)$ |
| Endpoint derivatives 1–3 at both ends | 6 |
| Total | $8k$ |

### C. Adding a specified final velocity

The fourth sub-question also requires $P'(1) = -2$. The equality constraints now read

$$A_N = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 2 & \cdots & N \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ -2 \end{bmatrix}. \tag{6}$$

For degree two, $A_2\boldsymbol{p} = b$ fixes $\boldsymbol{p} = (4, -3)^T$; the unique feasible path and its cost are

$$\boxed{P_2^*(t) = 4t - 3t^2, \qquad J_2^* = 4.} \tag{7}$$

For degree three, set $p_3 = z$. The constraints imply $p_1 = 4 + z$ and $p_2 = -3 - 2z$; substitution into the integral gives $J = 4 + z + 2z^2/15$. Differentiation yields $z = -15/4$ and

$$\boxed{P_3^*(t) = \tfrac{1}{4}t + \tfrac{9}{2}t^2 - \tfrac{15}{4}t^3, \quad J_3^* = \tfrac{17}{8}.} \tag{8}$$

The cubic accommodates the negative terminal velocity with a smaller integral cost than the constrained quadratic. Figure 1 illustrates how the terminal-velocity constraint changes the position and velocity profiles.

## III. Deliverable 2: Minimum-Snap Segments

### A. Degree and number of independent conditions

For $k$ polynomial segments with fixed knot times $t_0 < \cdots < t_k$, the minimum-snap functional is

$$J = \sum_{i=1}^{k} \int_{t_{i-1}}^{t_i} \left[P_i^{(4)}(t)\right]^2 dt. \tag{9}$$

The interior Euler–Lagrange condition is $P_i^{(8)} = 0$. Thus a stationary extremal has degree at most seven and eight coefficients per segment, amounting to $8k$ scalar unknowns.

The required $8k$ equations can be accounted for explicitly: The interior continuity of velocity, acceleration and jerk establishes admissibility. Continuity of derivatives four through six arises from the natural stationarity conditions when intermediate derivatives are free. The seventh derivative may jump at an intermediate fixed-position waypoint. For $k = 3$, the accounting is $6 + 12 + 6 = 24$ equations for 24 coefficients.

### B. General derivative-order cost

For $\sum_i \int (P_i^{(r)})^2 dt$, the stationary pieces satisfy $P_i^{(2r)} = 0$: the degree is at most $2r - 1$, with $2rk$ coefficients. Endpoint position plus the two sides of the $k - 1$ intermediate positions supply $2k$ equations. Fixed orders 1 through $r - 1$ at the start and finish provide $2(r - 1)$. Matching orders 1 through $2r - 2$ at each interior knot gives $(2r - 2)(k - 1)$. Indeed,

$$2k + (2r - 2)(k - 1) + 2(r - 1) = 2rk. \tag{10}$$

Integrating the first variation $r$ times reveals the interior boundary terms

$$\sum_{j=0}^{r-1} (-1)^j P_i^{(r+j)} \delta P_i^{(r-1-j)}. \tag{11}$$

The waypoint position is fixed, so $\delta P = 0$; the free common variations of orders 1 to $r - 1$ yield the natural matching conditions at orders $r$ through $2r - 2$. This calculation also explains why the polynomial construction balances smoothness with the number of free knot derivatives.

The team implementation uses the library's $N = 10$-coefficient polynomial representation and `makeStartOrEnd(..., SNAP)`, which additionally fixes the endpoint snap under that library convention. This numerical parameterisation is richer than the minimal degree-seven extremal used to answer the theoretical count.

## IV. Deliverable 3: From Gates to Rotor Commands

### A. Waypoint data and the first-gate test

The race-course gate positions and quaternions are read from Unity's static-transform CSV, converted to the ROS coordinate convention, sorted by gate number, and published as a `geometry_msgs/PoseArray` on `/desired_traj_vertices`. The first implementation checkpoint uses `simple_traj_planner.cpp` to publish the first gate as a stationary reference: zero linear velocity and acceleration permit a reproducible hover test before full racing.

The final architecture is shown in Fig. 2. The trajectory node creates a reference and forwards an odometry sample through a synchronised topic; the shared Lab 3 geometric controller [3] converts position and attitude errors to rotor commands. ROS integration and simulator connection were completed jointly by the group.

### B. Position, heading and snap-optimal references

Following the minimum-snap formulation [2], the waypoint generator optimises $(x, y)$, vertical position $z$, and unwrapped yaw $\psi$ in separate polynomial systems with shared segment durations. Position is prescribed at the gates; yaw is unwrapped by

$$\psi_i = \psi_{i-1} + \operatorname{atan2}\Big(\sin(\hat{\psi}_i - \psi_{i-1}), \cos(\hat{\psi}_i - \psi_{i-1})\Big), \tag{12}$$

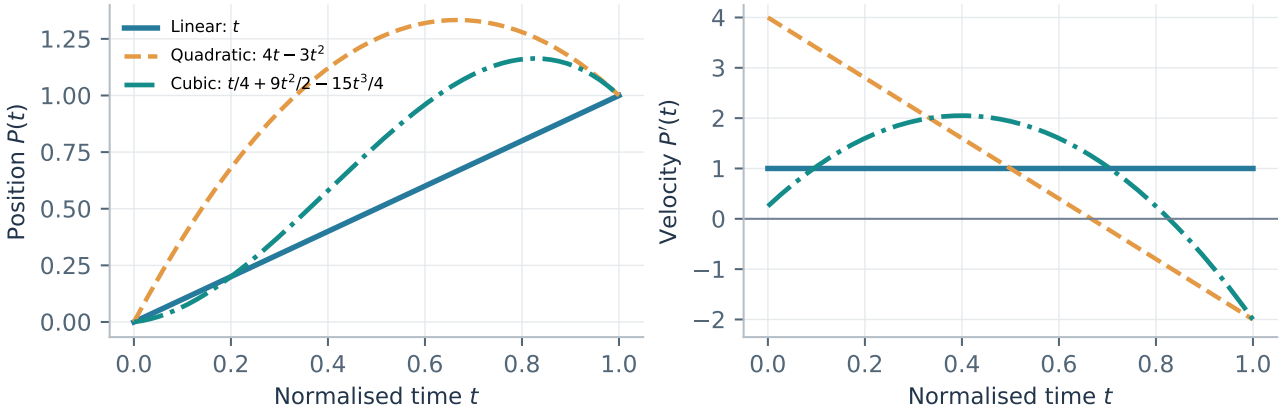



Fig. 1. Position and velocity profiles for the three analytic solutions. The linear curve solves the original fixed-position problem; the quadratic and cubic curves solve the $P'(1) = -2$ variant. The figure uses the exact expressions in (7)–(8).

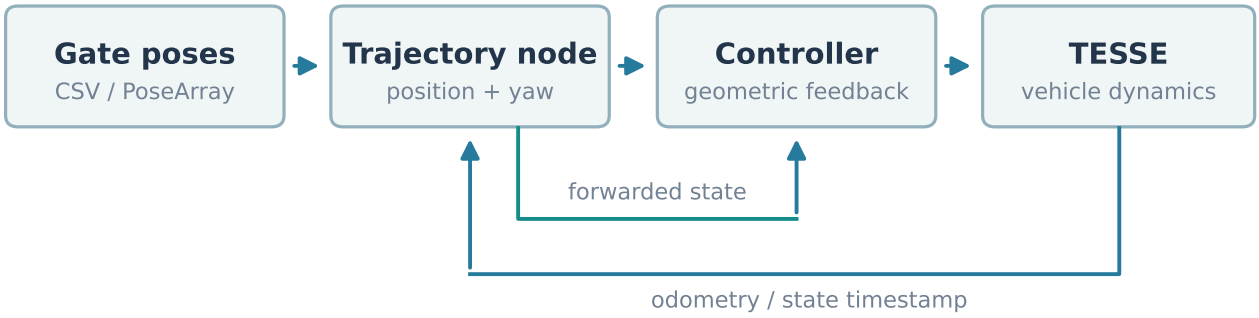



Fig. 2. Communication and control flow reconstructed from the implemented nodes. The state timestamp drives the trajectory sample in the final configuration. The feedback branch carries the measured state back to the reference-sampling and control stages.

so successive headings follow the shorter turn around the $\pm\pi$ branch cut. Start/end vertices constrain derivatives using the planning library's snap convention, and intermediate vertical velocities are set to zero at each gate. The solver evaluates the polynomial and its derivatives at time $t$ to publish the desired pose, velocity, acceleration and yaw rate. At the finish, it continues to publish the terminal rest pose.

### C. Per-segment time allocation and spatial constraints

The snap objective controls trajectory smoothness at fixed segment times. The time-allocation stage sets the racing pace. The implemented planner begins with distance-based segment-time estimates and iterates per-segment scaling using sampled estimates of speed $\|\boldsymbol{v}\|$, total acceleration $\|\boldsymbol{a}\|$, and vertical acceleration $|a_z|$. The selected parameters are $v_{\max} = 8\,\mathrm{m/s}$, $a_{\max} = 6\,\mathrm{m/s^2}$, and $a_{z,\max} = 5\,\mathrm{m/s^2}$.

A 0.5 m *corridor* parameter limits the additional deviation of each new segment from the chord connecting its end points, relative to the initial uniform-time trajectory. This protects the curve shape from large excursions when segment durations differ substantially. The vertical-velocity gate condition addresses overshoot caused by climbing momentum. The implementation evaluates velocity and acceleration at 101 equally spaced points per segment. For segment $i$, its sampled load is

$$\ell_i = \max_{t \in \mathcal{S}_i} \left\{ \frac{\|\boldsymbol{v}(t)\|}{v_{\max}}, \sqrt{\frac{\|\boldsymbol{a}(t)\|}{a_{\max}}}, \sqrt{\frac{|a_z(t)|}{a_{z,\max}}} \right\}. \tag{13}$$

For a fixed curve stretched in time by $s$, velocity scales by $s^{-1}$ and acceleration by $s^{-2}$; these relations explain the square roots in (13). Each iteration proposes $T_i \leftarrow T_i \ell_i^{\alpha}$, starting with $\alpha = 0.5$, and resolves the polynomials. The chord-deviation check uses 99 interior samples. A corridor violation restores the previous allocation and halves $\alpha$. The planner retains the shortest accepted allocation after scaling it to the sampled limits.

### D. Synchronising the reference with measurement time

TESSE state timestamps can advance in uneven simulation increments. Sampling the desired state on an unrelated timer may introduce an artificial discrepancy if the controller compares it with a state from a different simulation instant. With `sync_to_state=true`, the trajectory node generates a reference at each arriving

TABLE II
RECORDED PERFORMANCE OF THE TWELVE-GATE RACE.

| Performance measure | Observed |
|---|---|
| Prescribed gate count | 12 |
| Planned traversal time | 18.907 s |
| Tracking RMS, active flight | 0.06272 m |
| Largest tracking deviation | 0.11346 m |
| Largest nearest-gate distance | 0.07711 m |
| Final position error | 0.01692 m |
| End-position hold | 10.86 s |
| High-frequency tilt RMS | 0.0982° |
| Collision record | Floor at take-off only |

odometry timestamp and only then forwards that odometry as `/synced_state`. This publication sequence associates the reference calculation with the measurement time used by feedback. The timer remains available as a fallback.

## V. Experimental Results and Validation

### A. Recorded run and performance definitions

The experiment ran in Unity/TESSE with ROS Noetic on Ubuntu 20.04. The analysis uses the `unity-gpu-rviz-20261020` flight record [4], comprising 1,489 telemetry rows, the ROS bag, a saved summary and two simultaneous video recordings. The position gains were $k_x = 14$, $k_v = 6$; the attitude gains were $k_R = 12$, $k_\Omega = 2.2$. The reference lasted 18.9072 s, followed by terminal holding. Figures use the full CSV; Table II reports the saved run-summary statistics. A position error at an aligned sample is $e(t) = \|\boldsymbol{x}(t) - \boldsymbol{x}_d(t)\|_2$, with RMS

$$E_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{j=1}^{N} \|\boldsymbol{x}_j - \boldsymbol{x}_{d,j}\|_2^2}. \tag{14}$$

For gate centre $\boldsymbol{g}_i$, the recorded proximity is

$$d_i = \min_j \|\boldsymbol{x}_j - \boldsymbol{g}_i\|_2, \qquad \tau_i = t_{\arg\min_j \|\boldsymbol{x}_j - \boldsymbol{g}_i\|_2}. \tag{15}$$

The sequence $\tau_0 < \cdots < \tau_{11}$ records the order of closest approaches. Trajectory error measures deviation from the timed reference, while $d_i$ measures proximity to a fixed gate centre. The Unity gate-colour transitions provide the corresponding passage events.

### B. Traversing the course and maintaining accuracy

The planned and measured tracks in Fig. 3 nearly overlap around turns and height changes. All twelve gate-centre visits are ordered, and the largest recorded nearest-centre distance is 7.71 cm, occurring at gate 2. The gate-by-gate distribution in Fig. 5(a) distinguishes the more demanding portions of the circuit from gates with greater positional margin. The simulator recording shows gates changing colour as the drone traverses the route, complementing the quantitative centre-distance test.

The closest-approach intervals in Fig. 5(b) show how flight time is distributed along the course. Gate 0 is approached at approximately 2 s. The gate 9–10 interval lasts 2.50 s and the gate 10–11 interval lasts 2.45 s, compared with approximately 1.2–1.5 s for most intermediate intervals. These measurements direct attention to the final approach when evaluating local timing changes.

Figure 5(c) shows how tracking error varies across the curved flight segments. The final position error is about 1.69 cm and the terminal pose is maintained for 10.86 s. These results indicate an accurate completion of the current course under the selected controller and planning parameters. The collision log records floor contact during take-off and a collision-free airborne traversal.

### C. Selecting the racing parameters

The development records compare two nearby operating points with the same position and attitude gains: $v_{\max}/a_{\max} = 8/6$ and $9/7$, expressed in m/s and m/s$^2$, respectively. The first setting produced a trajectory of approximately 18.9 s and a maximum gate-centre distance of 0.074 m in the tuning runs. The second shortened the trajectory to approximately 17.8 s and increased that distance to 0.089 m. These tuning runs informed the choice of 8/6 for the final recording. The final run then measured 0.0771 m as its maximum gate-centre distance. Holding the gains fixed makes this local comparison useful for choosing a practical speed setting.

### D. Flight recordings and data

The Unity and RViz recordings cover the same 32.1 s session, including traversal and terminal holding. Figure 4 shows frames at 00:07 and 00:24. Unity displays the gate-colour transitions, while RViz displays planned and actual paths and gate orientations. The recorded run folder contains the ROS bag, complete CSV, configuration and summary. Direct video links appear at the end of this report.

## VI. Individual Discussion

### A. Gate-centre accuracy and local timing

My analysis begins with the spatial distribution of the error. Gate 2 has the largest centre distance, 77.1 mm, followed by gate 10 at 74.1 mm and gate 4 at 73.4 mm. Gate 11 is approached within 15.2 mm. This pattern identifies where local timing and heading deserve attention. The centre-distance measurements describe approach accuracy; the physical clearance also depends on gate dimensions, vehicle size and orientation.

The final two approach intervals are the longest after take-off. Their durations reflect the combination of course geometry and the terminal rest condition. A useful next experiment is to vary one neighbouring segment time at a time while retaining the same controller gains and initial state. Recording centre distance, crossing heading and peak acceleration for each gate would show which local adjustment reduces total time while preserving accurate passage.

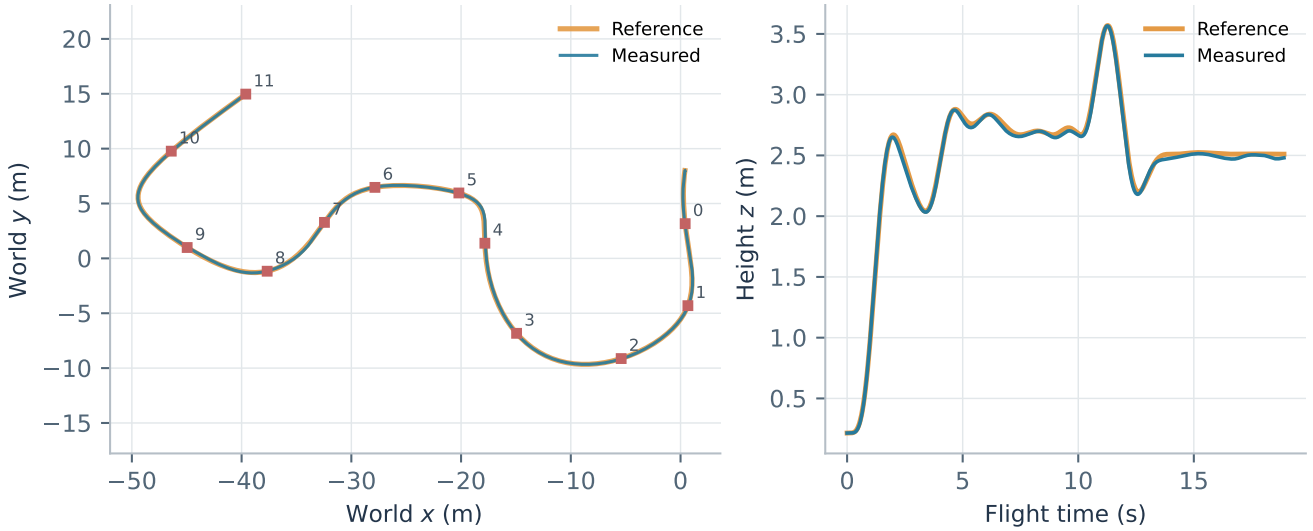
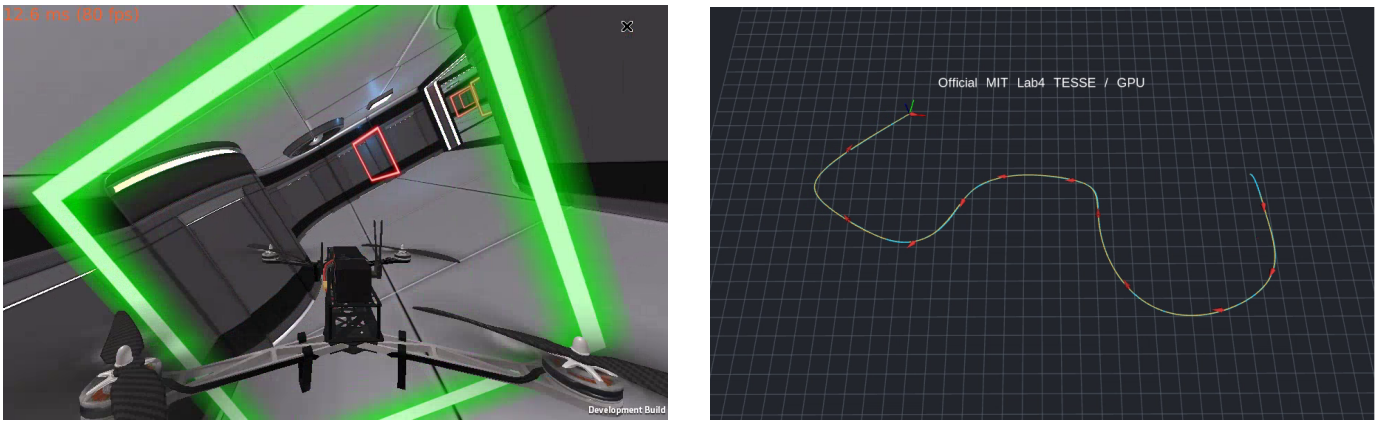



Fig. 3. World-coordinate racing trajectory from the full flight CSV. Left: reference and measured paths with the twelve gate centres from the course transform file. Right: reference and measured height during traversal.




(a) Unity simulator, 00:07

(b) RViz trajectory display, 00:24

Fig. 4. Frames from the recorded Unity and RViz demonstrations at the indicated playback times. Unity shows a green gate during traversal; the cropped RViz viewport shows the reference, measured path and gate-direction markers.

### *B. Reference timing and tracking error*

At 8 m/s, a reference offset of 20 ms represents 0.16 m of travel. This is approximately 2.6 times the final run's 62.7 mm RMS tracking error. The calculation explains why timestamp alignment is a central part of this system: even a small temporal offset can create a substantial position discrepancy before the control law acts.

The implemented callback samples the trajectory using the incoming odometry timestamp and then publishes that odometry to the controller. For further analysis, I would log the source timestamp of the state, reference and resulting command together. Their differences would quantify message alignment, and a lag analysis of the aligned trajectories would characterise the vehicle's physical response. This separates two mechanisms that call for different changes to the software or controller.

### *C. Vertical motion and feasible acceleration*

The vertical channel links trajectory design directly to thrust. With upward world axis $\boldsymbol{e}_3$, thrust magnitude $f$ and body-to-world rotation $R$, the translational model gives

$$m\ddot{z} = f\,\boldsymbol{e}_3^T R\boldsymbol{e}_3 - mg. \tag{16}$$

During upright flight with positive thrust, reducing thrust provides downward acceleration, while tilting also allocates force to horizontal motion. A sharp climb followed by rapid braking therefore places coupled demands on vertical thrust and attitude. The separate $5\,\mathrm{m/s^2}$ vertical-acceleration setting and zero vertical velocity at each gate shape this transition explicitly.

Unequal segment times also change the polynomial geometry. The planner's corridor parameter bounds each candidate's additional departure from its waypoint chord relative to the initial curve. I would extend this evaluation

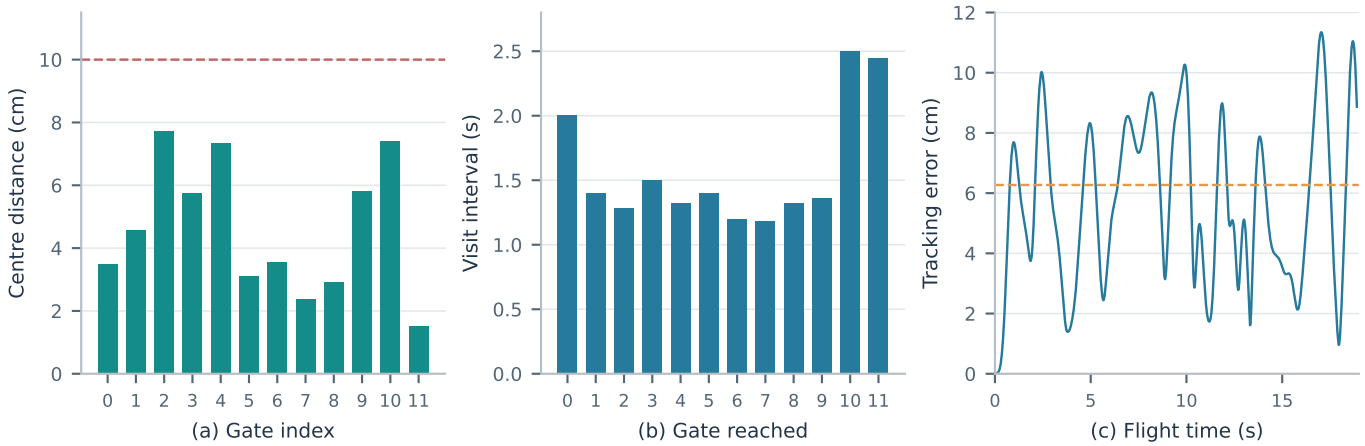



Fig. 5. Race diagnostics. (a) Nearest gate-centre distances, with a 10 cm comparison threshold. (b) Intervals between successive closest approaches, including departure to gate 0. (c) Tracking error from every recorded state/reference pair during traversal; the dashed line marks the RMS reported in the saved summary.

by recording thrust, tilt and rotor-speed extrema alongside the sampled trajectory limits. Repeated runs at fixed initial conditions would then relate faster timing to the actual actuator demands and passage accuracy.

### D. Gate-plane events for automated evaluation

A gate-plane event detector would add crossing time and orientation to the existing proximity measurements. For centre $\boldsymbol{g}_i$ and unit normal $\boldsymbol{n}_i$, define

$$s_i(t) = \boldsymbol{n}_i^T \big(\boldsymbol{x}(t) - \boldsymbol{g}_i\big). \tag{17}$$

A sign change in the required direction identifies a candidate crossing. Interpolation between the adjacent samples gives its position and time. Projecting that point onto the gate's two in-plane axes then tests the rectangular opening with a margin for vehicle size. Applying these tests in gate order would produce an automatic lap record containing crossing time, heading and clearance for all twelve gates.

## VII. Conclusion

The single-segment derivation expresses velocity-energy minimisation as a quadratic program and yields the optimal linear, quadratic and cubic solutions under the prescribed boundary conditions. The multi-segment derivation explains the $8k$ conditions for minimum-snap motion and their extension to an $r$-th derivative objective. These principles support the implemented position and yaw planner, which combines segment-time refinement, vertical-motion constraints and state-timestamp sampling with geometric feedback.

The recorded race traversed twelve gates with a planned duration of 18.907 s, a reported position-tracking RMS of 62.7 mm and a maximum nearest-gate-centre distance of 77.1 mm. The vehicle finished with a 16.9 mm position error and held the terminal pose for 10.86 s. My evaluation identifies local gate accuracy, time alignment and actuator demand as the quantities to track when refining the racing pace.

## Source Code and Recordings

**Individual D1 and D2 -- unified report**
https://github.com/Moln-ya/vnav-personal/blob/main/lab4/lab4_junxi_LI.pdf
**Team Lab4 -- planner and flight data**
https://github.com/chixin114514/2026Fall_TeamFullcans/tree/main/lab4
**Unity race recording**
https://github.com/chixin114514/2026Fall_TeamFullcans/blob/main/lab4/report/video/unity_gpu_race.mp4
**RViz race recording**
https://github.com/chixin114514/2026Fall_TeamFullcans/blob/main/lab4/report/video/rviz_gpu_race.mp4
**Team video-path notes**
https://github.com/chixin114514/2026Fall_TeamFullcans/blob/main/lab4/rviz_drone_race.txt

## REFERENCES

[1] MIT VNAV, "Lab 4: Exercises," 2023. [Online]. Available: https://vnav.mit.edu/labs_2023/lab4/exercises.html.

[2] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in Proc. IEEE Int. Conf. Robotics and Automation, 2011, pp. 2520-2525.

[3] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in Proc. 49th IEEE Conf. Decision and Control, 2010, pp. 5420-5425.

[4] Group 6, "Lab 4: Polynomial planning, race telemetry, controller integration and recordings," 20 October 2026. https://github.com/chixin114514/2026Fall_TeamFullcans/tree/main/lab4